Traffic Light Management System

Two crossing roads - timer interrupt - C language

1. Introduction

The goal of this project is to simulate traffic lights for two crossing roads (Road_A and Road_B). Only one road has a green light at a time while the other is red, and the roles alternate. The program is written in C, displays the state of each light with ANSI colors, and counts the vehicles that pass on the green road. The light timing is driven by a **timer interrupt**.

2. Data Structure

```
struct Road {
    char name[20];  // road name
    int remain;     // cars currently waiting
    int number;     // initial number of cars (used to refill the queue)
};
```

Road_A starts with 50 cars and Road_B with 40. The variables `road_green` and `road_red` hold the road that is currently green and the one that is red. They are swapped at the end of every cycle.

3. The Timer Interrupt

3.1 Principle

A timer interrupt is an event fired at regular intervals by a timer, independently of the main program. Each time it fires, it runs a short function called the handler (ISR, Interrupt Service Routine). The main program does not decide when time has passed: it reacts to the timer. The previous version used `sleep(1)`, which is only a blocking wait and not an interrupt.

3.2 Common idea in both systems

Both versions give the main code the same two functions: `start_timer()` (starts a 1-second timer) and `wait_tick()` (waits for the next timer tick, replaces `sleep(1)`). The handler `timer_isr` only signals the tick, because an interrupt handler must stay very short. The code is selected at compile time with `#ifdef _WIN32 ... #else ... #endif`.

3.3 Linux implementation (SIGALRM + setitimer)

Item	Role
`tick` (volatile sig_atomic_t)	Flag: 0 = no tick yet, 1 = the timer just fired. *sig_atomic_t* can be written safely inside a handler; *volatile* stops the compiler from caching it.
`timer_isr(int sig)`	The interrupt handler. Called by the kernel at every SIGALRM. Only sets tick = 1.
`wait_tick()`	Blocks SIGALRM, then sleeps with sigsuspend until the signal arrives, clears the flag and restores the mask. Blocking first guarantees no tick is lost.
`sigset_t block, old`	block = set containing SIGALRM; old = saved previous mask.
`sigemptyset / sigaddset`	Empty a signal set / add a signal to it.
`sigprocmask`	Blocks or restores signals for the process.
`sigsuspend(&old)`	Atomically unblocks signals and sleeps until one arrives.
`struct sigaction sa` + `sigaction`	Installs timer_isr as the handler of SIGALRM.
`struct itimerval t`	Timer description: it_value = first delay (1 s), it_interval = period (1 s).
`setitimer(ITIMER_REAL, &t, NULL)`	Starts the real-time timer; the kernel sends SIGALRM at each expiration.

3.4 Windows implementation (timer queue + event)

Item	Role
`tick_event` (HANDLE)	A Windows event object, either signaled or not signaled. Auto-reset.
`timer_isr(...)`	Callback called every 1000 ms by the timer. Only calls SetEvent.
`wait_tick()`	WaitForSingleObject(tick_event, INFINITE) blocks until the event is signaled.
`hTimer`, `hOut`, `mode`	Handle of the timer, handle of the console output, and console flags (DWORD).
`GetConsoleMode / SetConsoleMode`	Adds flag 0x0004 (ENABLE_VIRTUAL_TERMINAL_PROCESSING) so the console shows ANSI colors.
`CreateEvent(NULL, FALSE, FALSE, NULL)`	Creates the event: default security, auto-reset, initially not signaled, no name.
`CreateTimerQueueTimer(...)`	Creates the timer: callback timer_isr, first delay 1000 ms, period 1000 ms.

3.5 Remark

On both systems this is a software-level interrupt managed by the operating system, driven by the hardware timer. On a microcontroller (Arduino, STM32...), the same `timer_isr` would be attached directly to a hardware timer interrupt.

4. Program Logic

main(): initializes the two roads (Road_A green, Road_B red), calls `start_timer()`, then loops forever: run one cycle with `passe_car()`, swap the green and red roads, and refill a road with `number` cars if its queue reached 0.

passe_car() runs one full cycle:

Phase	Duration	What happens
Green / Red	60 ticks	Countdown displayed at each tick. One car passes every 3 ticks (i % 3 == 0), so at most 20 cars per cycle.
Yellow	10 ticks	Warning phase, no car passes.
Switch	instant	Roles of the two roads are swapped in main().

The number of waiting cars decreases only on the green road. A long queue (Road_A has 50 cars) needs several cycles to empty. When a queue reaches zero it is refilled to simulate new traffic.

5. Compilation and Execution

On Linux (Pop!_OS, Ubuntu, Debian...)

```
gcc -Wall main.c -o main
./main
```

To stop the program press **Ctrl + C**.

On Windows

With MinGW (gcc) installed, open cmd or PowerShell in the folder of main.c:

```
gcc main.c -o main.exe
main.exe
```

In PowerShell use `.\main.exe`. If you only received the compiled file, just open cmd in the folder and type `main.exe`. Press **Ctrl + C** to stop. Colors require Windows 10 or later.

Cross-compiling the Windows .exe from Linux

```
sudo apt install mingw-w64
x86_64-w64-mingw32-gcc main.c -o main.exe -static
```

The result is a Windows program. It cannot run directly on Linux, but it can be tested with Wine: `sudo apt install wine64` then `wine main.exe`.

System	Compile	Run
Linux	gcc -Wall main.c -o main	./main
Windows (cmd)	gcc main.c -o main.exe	main.exe
Windows (PowerShell)	gcc main.c -o main.exe	.\main.exe
Linux to Windows .exe	x86_64-w64-mingw32-gcc main.c -o main.exe -static	wine main.exe (test only)

6. Conclusion

The program simulates two crossing roads with green, yellow and red phases and counts the vehicles passing on the green road. Its timing is driven by a timer interrupt (SIGALRM with setitimer on Linux, a timer queue callback on Windows) that fires every second, so each step of the simulation advances exactly once per timer tick.